Objetivo y reglas del Tres en Raya

Implementar y verificar las reglas del juego mediante pruebas unitarias reproducibles.

1. Tablero 3x3. Coordenadas de 1 a 3 en ambos ejes.
2. X inicia y alterna con +. Una casilla ocupada o fuera del tablero produce una excepción.
3. Gana quien completa una fila, columna o diagonal. Sin una línea completa se informa No hay ganador.

Fuente: material TDD del docente, páginas 16-18. Referencia: github.com/gpalabral/TDD.

Herramientas, API y estructura

Elemento	Configuración
Lenguaje y build	Código compatible con Java 8; Maven; JUnit 4.13 en pom.xml.
API	com.ucb.tdd.TresEnRaya: jugar(int x,int y) y siguienteTurno().
Programa	src/main/java: motor TresEnRaya y consola Main.
Pruebas	src/test/java: base, ampliada, docente, guiados y regresiones separadas.
Automatización	GitHub Actions configurado con Maven y JDK 17.

Fuente: pom.xml, árbol src y .github/workflows/maven.yml.

Proceso TDD y contribuciones

1. El historial conserva 16 ciclos anteriores: prueba fallida, implementación suficiente y comprobación de regresiones.
2. La refactorización separó validación y detección de líneas. Las suites comparadas se añadieron después del motor: son verificaciones posteriores.
3. La base se guardó antes de generar la ampliada. Ambas suites utilizan el mismo motor, API y reglas, sin cambios para favorecer resultados.

El programa y ambas suites recibieron asistencia de IA. Se conservan las pruebas del docente por separado. La comparación literal de autoría humana independiente frente a IA sigue pendiente.

Fuentes: historial Git, AUTORIA.md y huellas de evidencia/actual/resultados.json.

Suite base y suite ampliada

Dimensión	Base	Ampliada
Organización	28 métodos explícitos, preparación/acción/comprobación	16 métodos con datos agrupados y un ayudante
Coordenadas	0 y 4 en ambos ejes	Añade extremos int y dos estados de turno
Errores	Excepción y conservación del turno	Añade continuidad del tablero y rechazos repetidos
Victoria	8 líneas para X y 8 para +	Mismas líneas, respuestas y turnos intermedios
Escenarios extra	Reglas obligatorias del curso	Consultas, independencia, finales y tablero lleno
Mantenimiento	Lectura directa, mayor repetición	Menos repetición, mayor nivel de abstracción

Fuente: TresEnRayaBaseTest.java y TresEnRayaIATest.java. Métodos y escenarios no son equivalentes.

Ejemplo 1: estado después de un error

Base: excepción y turno

```
juego.jugar(2, 2);
assertThrows(Exception.class,
    () -> juego.jugar(2, 2));
assertEquals('+',
    juego.siguienteTurno());
```

Ampliada: continuidad del tablero

```
// Tras rechazar una ocupada:
juego.jugar(2, 1);
juego.jugar(1, 2);
juego.jugar(2, 2);
assertEquals("X es el ganador",
    juego.jugar(1, 3));
```

La ampliada utiliza una victoria posterior para comprobar que la ficha inicial no se borró al rechazar la jugada. La base observa directamente la excepción y el turno.

Fragmentos de casillaOcupada... (base) y rechazarOcupadaNoBorra... (ampliada).

Ejemplo 2: comprobación de una victoria

Base: secuencia concreta

```
juego.jugar(1, 1);
juego.jugar(2, 1);
juego.jugar(1, 2);
juego.jugar(2, 2);
String resultado = juego.jugar(1, 3);
assertEquals("X es el ganador",
    resultado);
```

Ampliada: cada paso de la partida

```
assertEquals(
    "Resultado del movimiento " + i,
    esperado, resultado);
assertEquals(
    "Turno posterior " + i,
    i%2==0 ? '+' : 'X',
    juego.siguienteTurno());
```

Ambas suites prueban las ocho líneas para cada jugador. La ampliada comprueba también las respuestas anteriores a la victoria y el turno tras cada movimiento.

Fragmentos de xGanaEnPrimeraFila (base) y comprobarPartida (ayudante de la ampliada).

Resultados de la versión definitiva

Ejecución	Métodos	Java 8	JDK 17
Base	28	PASS	PASS
Ampliada	16	PASS	PASS
Base y ampliada juntas	44	PASS	PASS
Todas las suites	75	PASS	PASS

Compilación con javac 17.0.6 y --release 8, bytecode major 52. Ejecución con JRE 1.8.0_241 y JDK 17.0.6, utilizando JUnit local 4.13.2. El JAR funciona en ambos runtimes.

Fuente: evidencia/actual/resultados.json y logs nuevos de suites, consola y JAR.

Diferencias observadas en copias de prueba

Defecto introducido temporalmente	Base	Ampliada
M1: borra una ficha antes de rechazar	No detecta	Detecta
M2: omite la diagonal opuesta	Detecta	Detecta
M3: consume turno al rechazar ocupada	Detecta	Detecta

La base es suficiente para explicar las reglas obligatorias con casos concretos. La ampliada observa más efectos posteriores. Tener más métodos no garantiza mayor calidad.

Los tres defectos son una muestra ilustrativa. Se aplicaron en copias temporales, sin modificar el motor entregado.

Fuente: evidencia/actual/M1-*, M2-* y M3-*. Cambios exactos en scripts/verificar_final.py.

Conclusiones y límites de verificación

1. Las reglas del juego y la compatibilidad con las 10 pruebas del docente se verificaron localmente. No se midió cobertura de líneas ni se demuestra corrección total.
2. Maven 3.9.16 se instaló localmente. Su descarga de dependencias está bloqueada por permisos de red; las pruebas locales usan JUnit 4.13.2.
3. GitHub Actions verificó Maven y JUnit 4.13: base 28, ampliada 16, juntas 44 y conjunto completo 75; cero fallos y errores.

Repositorio público: github.com/diegocoaquira-source/taller6-tdd-tres-en-raya

Fuentes: evidencia/actual/github-actions.txt, ejecución 40772099194; referencia github.com/gpalabral/TDD.